Module 6 -- Robotics Projects

Lesson 44: Robot Movement Functions

★ Star-5 Expert | ⏰ 45--55 minutes | 👤 Age 6--12 years | ⚡ 330 XP

Your Mission Today

Robot Programmer, your robot can move — but right now you have to write the same motor commands over and over every time you want it to do something. That is messy and slow! Today you are going to write **movement functions** — reusable blocks of code that let you command your robot with simple words like `forward()`, `turnLeft()`, and `stopRobot()`. Then you will program your robot to drive in a perfect square and even a figure-8!

Learning Objectives

By the end of this lesson, you will be able to:

- Write reusable Arduino functions for robot movement
- Program the robot to follow a planned path (square, figure-8)
- Adjust motor speeds to make the robot drive straight
- Use your Magic Measurement Wand to measure voltage on motor driver output pins during forward, backward, and stop

What You Need

Item	Qty
Your assembled robot from Lesson 43	1
Arduino Uno (on robot)	1
USB cable (for programming)	1
Multimeter (your Magic Measurement Wand!)	1
Measuring tape or ruler	1
Masking tape (to mark floor path)	1 roll

How to Teach This Lesson

Step 1: Hook — The Lazy Programmer (5 min)

Show the code from Lesson 43's test sketch:

> "Look at all this code just to go forward. What if I want the robot to go forward, turn right, go forward, turn left, go forward, and stop? I would have to copy and paste all those motor pin commands EVERY SINGLE TIME. That is like writing the entire recipe for a sandwich every time you want lunch instead of just saying 'make me a sandwich.'"

> "What if you could just write `forward(200)` and the robot KNOWS what to do? That is what functions are! They are shortcuts that make your code clean and simple."

Award: **+10 XP** **for understanding why functions matter!**

Step 2: Writing Movement Functions (12 min)

The Complete Robot Movement Library:

```
// Lesson 44: Robot Movement Functions
// A clean library of movement commands

// === MOTOR PIN DEFINITIONS ===
int leftENA = 9;    // Left motor speed (PWM)
int leftIN1 = 2;    // Left motor direction
int leftIN2 = 3;    // Left motor direction
int rightENB = 10;  // Right motor speed (PWM)
int rightIN3 = 4;   // Right motor direction
int rightIN4 = 5;   // Right motor direction

// === SETUP ===
void setup() {
  pinMode(leftENA, OUTPUT);
  pinMode(leftIN1, OUTPUT);
  pinMode(leftIN2, OUTPUT);
  pinMode(rightENB, OUTPUT);
  pinMode(rightIN3, OUTPUT);
  pinMode(rightIN4, OUTPUT);
  Serial.begin(9600);
  Serial.println("Robot Movement System Ready!");
  delay(2000);
}

// === MOVEMENT FUNCTIONS ===

void forward(int speed) {
  digitalWrite(leftIN1, HIGH);
  digitalWrite(leftIN2, LOW);
  digitalWrite(rightIN3, HIGH);
  digitalWrite(rightIN4, LOW);
  analogWrite(leftENA, speed);
  analogWrite(rightENB, speed);
  Serial.print("Forward at speed: ");
  Serial.println(speed);
}

void backward(int speed) {
  digitalWrite(leftIN1, LOW);
  digitalWrite(leftIN2, HIGH);
  digitalWrite(rightIN3, LOW);
  digitalWrite(rightIN4, HIGH);
  analogWrite(leftENA, speed);
  analogWrite(rightENB, speed);
  Serial.print("Backward at speed: ");
```

```
}

void turnLeft(int speed) {
  digitalWrite(leftIN1, LOW);
  digitalWrite(leftIN2, HIGH);   // Left wheel backward
  digitalWrite(rightIN3, HIGH);
  digitalWrite(rightIN4, LOW);   // Right wheel forward
  analogWrite(leftENA, speed);
  analogWrite(rightENB, speed);
  Serial.println("Turning LEFT");
}

void turnRight(int speed) {
  digitalWrite(leftIN1, HIGH);
  digitalWrite(leftIN2, LOW);    // Left wheel forward
  digitalWrite(rightIN3, LOW);
  digitalWrite(rightIN4, HIGH);  // Right wheel backward
  analogWrite(leftENA, speed);
  analogWrite(rightENB, speed);
  Serial.println("Turning RIGHT");
}

void stopRobot() {
  digitalWrite(leftIN1, LOW);
  digitalWrite(leftIN2, LOW);
  digitalWrite(rightIN3, LOW);
  digitalWrite(rightIN4, LOW);
  analogWrite(leftENA, 0);
  analogWrite(rightENB, 0);
  Serial.println("STOPPED");
}

// === MAIN LOOP ===
void loop() {
  // Simple test: forward, pause, backward, pause
  forward(180);
  delay(2000);
  stopRobot();
  delay(1000);
  backward(180);
  delay(2000);
  stopRobot();
  delay(3000);
}
```

> 💬 "See how clean the loop() is now? Just ONE LINE to go forward instead of six lines of motor commands. That is the power of functions!"

Walk through the code together:

- Each function takes a `speed` parameter (0-255)
- The function handles ALL the pin settings internally
- The `loop()` just calls functions by name — easy to read!

Award: **+30 XP** **for writing the complete movement function library!**

Step 3: Hands-On — Drive in a Square (10 min)

Challenge: Program the robot to drive in a square. Use masking tape on the floor to mark a square path.

```
void loop() {
  // Drive in a square!
  for (int side = 0; side < 4; side++) {
    forward(180);        // Drive one side
    delay(1500);         // Adjust time for side length
    stopRobot();
    delay(500);
    turnRight(180);      // Turn 90 degrees
    delay(450);          // Adjust time for exact 90-degree turn
    stopRobot();
    delay(500);
  }
  stopRobot();
  delay(5000);  // Pause before repeating
}
```

Tuning Tips:

- Forward `delay(1500)` controls how FAR the robot goes per side
- TurnRight `delay(450)` controls how much it TURNS
- If the robot turns MORE than 90 degrees, decrease the turn delay

> 💬 "Getting the robot to make a perfect square takes TUNING. Real robotics engineers spend lots of time tuning! Keep adjusting until your robot ends up right back where it started."

Award: **+40 XP** **for making the robot drive in a recognizable square!**

Step 4: Hands-On — Drive in a Figure-8 (8 min)

The Figure-8 Challenge:

```
void loop() {
  // Figure-8: one circle left, one circle right

  // Circle 1: Turn left while moving forward
  Serial.println("Circle LEFT");
  digitalWrite(leftIN1, HIGH);
  digitalWrite(leftIN2, LOW);
  digitalWrite(rightIN3, HIGH);
  digitalWrite(rightIN4, LOW);
  analogWrite(leftENA, 120);   // Left wheel SLOWER
  analogWrite(rightENB, 200);  // Right wheel FASTER
  delay(4000);                 // Full circle time
  stopRobot();
  delay(500);

  // Circle 2: Turn right while moving forward
  Serial.println("Circle RIGHT");
  digitalWrite(leftIN1, HIGH);
  digitalWrite(leftIN2, LOW);
  digitalWrite(rightIN3, HIGH);
  digitalWrite(rightIN4, LOW);
  analogWrite(leftENA, 200);   // Left wheel FASTER
  analogWrite(rightENB, 120);  // Right wheel SLOWER
  delay(4000);                 // Full circle time
  stopRobot();
  delay(3000);
}
```

> 💬
> "A figure-8 uses GENTLE turns, not spinning in place. You make one wheel go faster than the other — like rowing a boat with one arm stronger than the other!"

Award: **+30 XP** **for making a figure-8!**

Step 5: Motor Speed Calibration (5 min)

> 💬
> "Does your robot drift to one side when you tell it to go straight? That is because no two motors are exactly the same. One might be a tiny bit faster. We can fix that!"

Calibration Method:

1. Place robot on the floor pointing at a target 1 meter away
2. Run `forward(200)` for 3 seconds
3. Does it drift left or right?
4. If it drifts RIGHT, the LEFT motor is faster — reduce left motor speed
5. If it drifts LEFT, the RIGHT motor is faster — reduce right motor speed

```
// Calibrated forward function with speed offset
int leftOffset = 0;   // Adjust this: negative = slower left
int rightOffset = 0;  // Adjust this: negative = slower right

void forwardCalibrated(int speed) {
  digitalWrite(leftIN1, HIGH);
  digitalWrite(leftIN2, LOW);
  digitalWrite(rightIN3, HIGH);
  digitalWrite(rightIN4, LOW);
  analogWrite(leftENA, speed + leftOffset);
  analogWrite(rightENB, speed + rightOffset);
}

// Try values like leftOffset = -15 or rightOffset = -10
```

Award: **+20 XP** **for calibrating your robot to drive straight!**

Step 6: Wand Check — Motor Driver Voltages (8 min)

"Time to peek under the hood with your Magic Measurement Wand! We are going to measure what the L298N motor driver is actually sending to your motors during each movement."

Set the Wand to **DC Volts**.

Measurement 1: Forward

Run the robot forward. Measure voltage across the left motor (OUT1 to OUT2):

```
| Robot State     | Left Motor Voltage | Right Motor Voltage |
|----------------|--------------------|---------------------|
| Forward (200)  |                    |                     |
| Backward (200) |                    |                     |
| Stop           |                    |                     |
| Turn Left (180)|                    |                     |
```

"When going forward, you should see about 4-5V across each motor. When stopped, you should see 0V. The polarity FLIPS when you go backward — your Wand might show a negative number. That just means current is flowing the other direction!"

Measurement 2: PWM Speed Effect

Run forward at different speeds and measure:

```
| PWM Speed | Expected Voltage | Measured Voltage |
|-----------|-----------------|-----------------|
| 100       | ~2.0V           |                 |
| 150       | ~3.0V           |                 |
| 200       | ~4.0V           |                 |
| 255       | ~5.0V           |                 |
```

"See? The PWM value controls the AVERAGE voltage the motor gets. Higher PWM = higher average voltage = faster spinning. Your Wand just proved it!"

Award: **+50 XP** **for all Wand measurements!**

Quick Quiz — Earn Bonus XP!

Question 1: What is the main benefit of writing movement functions?

- A) It makes the robot go faster
- B) It makes the code reusable, readable, and easy to change
- C) It uses less electricity

(Correct: B — +20 XP !)

Question 2: To make the robot turn right while driving (gentle curve), you should:

- A) Stop both motors
- B) Make the left wheel go faster than the right wheel
- C) Reverse both motors

(Correct: B — +20 XP !)

Question 3: Your robot drifts to the left when driving forward. Which motor is probably faster?

- A) The left motor
- B) The right motor
- C) Both are the same

(Correct: B — +20 XP !)

Lesson 44 Complete!

```
==============================================

   MOVEMENT MASTER BADGE UNLOCKED!

   Skills unlocked:
   [check] Wrote reusable movement functions
   [check] Drove in a square
   [check] Drove in a figure-8
   [check] Calibrated motor speeds
   [check] Measured driver output voltages

==============================================
```

XP Breakdown:

Activity	XP
Hook understanding	10
Movement function library	30
Square driving	40
Figure-8 driving	30
Motor calibration	20
Wand Check (all measurements)	50
Quiz (3 questions)	60
TOTAL POSSIBLE	**240**

Coming Up Next...

In **Lesson 45**, your robot gets EYES! You will add **IR line sensors** to the bottom of your robot so it can follow a black line on the floor — all by itself, with NO remote control. Your robot is about to become AUTONOMOUS!

Troubleshooting

Problem	Fix
Robot does not drive straight	Calibrate motor speeds using the offset method
Turn is not exactly 90 degrees	Adjust the turn delay time — smaller delay = less turn
Robot does not move at low PWM values	Motors need a minimum voltage — try PWM 120+
Code does not compile	Check for missing semicolons and matching curly braces
Serial Monitor shows garbled text	Make sure baud rate matches — Serial.begin(9600) and set Monitor to 9600
Wand shows negative voltage	Probe polarity is reversed — this is normal when measuring backward direction